Gagandeep Bhatia

Term Project: Instant Messaging App

Feature Document

Table of Contents

1 Introduction 2

1.1 Purpose 2

1.2 Target Audience 2

1.3 Terms and Definitions 2

2 Product Overview 3

2.1 Product Vision 3

2.2 Product Scope 4

2.3 Operating Environment 4

2.4 Design and Implementation Constraints 4

3 User Stories 6

3.1 Personas 6

3.2 User Scenarios 8

3.2.1 <User Scenarios with Persona 1> 8

3.2.2 <User Scenarios with Persona 2> ..... **Error! Bookmark not defined.**

3.2.3 <User Scenarios with Persona 3> 8

3.3 Epic User Stories 12

3.3.1 <Epic User Stories 1> 12

3.3.2 <Epic User Stories 2> 13

3.3.3 <Epic User Stories 3> 14

4 Feature List 16

1 Introduction

Provide an overview of this document and the product it describes.

1.1 Purpose

This document outlines the product, the users/target audience of this product and what features this product has to satisfy this target audience.

1.2 Target Audience

The target audience is students who need a messaging platform where they can share information about classes, events and resources.

1.3 Terms and Definitions

IDE: Integrated Development Environment

MERN stack:

- MongoDB: A database for storing and retrieving data.
- Express: A web application framework for building APIs.
- React: A JavaScript library for building user interfaces.
- Node.js: A backend framework for executing JavaScript

UI: User Interface

API: Application Programming Interface

2 Product Overview

This section describes who the product is for, what it does and why it is beneficial to the target audience. It also describes the features of the app, how the user will interact with the app, on what platforms this app will operate on and what technologies will be used to create this app with the described features from this section. It also covers potential constraints of the technology used to implement these features.

WHAT: An instant messaging app where users can send and receive messages in real time, and create group chats with multiple users. It focuses on the core features of communication rather than bells and whistles.

WHO: The product is aimed for students who need an easy-to-use communication platform.

WHY: This instant messaging app will allow students to communicate effectively with one another and share resources in real time. It will be developed and designed specifically for students who need to share information/resources with one another.

2.1 Product Vision

For students who need a messaging app. The instant messaging app is a web-based service that allows users to send messages, receive messages, create and delete chat rooms in real-time. Unlike Slack or discord, the product is tailored directly to students and integrates core functionality such as creating 1-1 chats, chats with many users, deleting chats and maintaining chat history.

2.2 Product Scope

2.3 Operating Environment

From a user perspective the instant messaging app requires a stable internet connection and a web browser (preferably Chrome) to operate.

From a developer perspective, the recommended developer environment is an IDE and to be in Windows, Mac or Linux with MongoDB as the database, Express as the framework for writing APIs, React for the frontend/UI and Node,js as the backend framework. Git will be used for version control. Additionally, the frontend will be provided by the TA.

2.4 Design and Implementation Constraints

The primary language that will be used to implement the features outlined in section 2.2 is JavaScript. The MERN tech stack will be used.

MongoDB will be used as the database for storing and updating the user's first and last names, storing user credentials, messaging history and created chat rooms along with their participants.

Express will be used to build APIs in JavaScript. It aids in creating a system that defines how an application responds to client requests at specific endpoints (URLs) and methods.

React is a JavaScript library that will be used to build the UI.

Node.js is a runtime environment that allows JavaScript code to be executed on a server.

For testing either the Jest or Mocha testing framework will be used.

A few constraints are:

- Project must be completed and full functional by March 17th
- Frontend is a black box since it will be provided by the TA
- The MERN stack might encounter performance issues if high traffic occurs
- The languages are limited to JavaScript and NoSQL (database)

3 User Stories

Understanding user needs is essential so that the desired functionality is integrated into the product. Once the desired functionality is understood, the development of backend APIs can begin so that the desired functionality is integrated. Additionally, understanding the target audience's familiarity with technology and instant messaging applications helps to design a better and more user-friendly application.

3.1 Personas

Persona 1: Alex Wood, 20 - Computer Science Major

Alex is a junior at Fresno State University studying computer science. He lives in an off-campus apartment with three roommates and also has a part-time job at the campus IT help desk. Between group projects, study sessions, and coordinating with TAs, Alex uses about four different messaging platforms throughout his day. He's comfortable with technology but dislikes the bloat that comes with enterprise tools like Slack, since he only uses a select few features.

Alex is seeking a simple communication platform for school. He needs to quickly create a group chat when he's assigned a team for class project, communicate with his physics classmate, and be able to look back at prior conversations as needed. He's not seeking integrations with other apps. Alex is looking for a platform where he can send messages in one place, organized by class and project, without having to pay for a premium tier or dealing with unnecessary features.

Persona 2: Priya Kaur, 21 - Data Science Major

Priya is a senior completing her bachelor's degree in data science from UC Berkeley with a concentration in machine learning. She has several collaborative commitments,

including a capstone research project analyzing predictive models, a statistical computing study group, and a data analytics team supporting a campus research initiative. Her technical background includes proficiency in Python, R, SQL, and various data visualization tools. Her current approach involves managing conversations across WhatsApp for personal connections, email for academic communication, and a Zulip workspace that lacks engagement from team members.

For academia, Priya is seeking a streamlined communication platform. She needs distinct chats for each project team where members can share dataset updates, discuss analytical approaches, and document decisions for future reference. As a Data Scientist student Priya revisits previous approaches and results often, thus she needs her chat history to be saved. She values platforms that are easy to use, have simple features and don't require extensive configuration or a steep learning curve. She is seeking a platform to consolidate academic communications in a clean, organized interface.

Persona 3: Matthew Edwards, 20 – Business Administration Major

Matthew is a junior studying business administration with a concentration in marketing at UT Austin. He serves as the social media coordinator for his school's entrepreneurship club and is currently enrolled in courses covering organizational behavior, market research, and strategic management. His skills are in business applications such as Microsoft Office, basic social media platforms, and consumer-level productivity tools. He currently communicates with team members for group projects through a combination of text messages, emails, and occasional video calls, which results in missed messages, confusion about meeting times, and difficulty locating shared information from previous discussions.

Matthew needs a communication tool that supports collaborative work and communication. Each semester involves multiple team projects with different classmates such as case study analyses, marketing campaign proposals, and business plan competitions; these all require coordination, draft reviews, and presentations. He finds that text message threads become cluttered and disorganized quickly, due to important

details being buried and he cannot access text messages from his PC. His ideal messaging platform would provide independent group chats for each project team, allowing him to reference past conversations when writing reports or preparing for presentations. He also wants to be able to delete group chats for projects at the end of the semester.

3.2 User Scenarios

3.2.1 User Scenarios with Persona 1: Alex Wood, 20 - Computer Science Major

Scenario 1A: Managing Multiple Project Teams

Overall Objective: Coordinate three simultaneous group projects across different courses clearly and accessible project history.

Alex has just started the fall semester and is enrolled in three courses: software engineering, machine learning, and a senior capstone project. Within the first two weeks, he's been assigned to three different project teams with students he's never worked with before. He currently uses Discord for one team, GroupMe for another, and a mix of text messages for the third. He misses important messages because notifications are scattered across platforms, and when he needs to reference a decision the team made two weeks ago about which API to use, he has to scroll through hundreds of messages mixed with off-topic conversations. His capstone team tried Slack, but half the members never log on, and the free version deletes old messages, which doesn’t allow him to refer back to important technical discussions from earlier in the semester.

Alex opens the student messaging app and creates three separate group chats: "CS401 - Web App Team," "ML Project - Neural Net," and "Capstone - Client Portal." He invites his teammates to each respective chat using their university email addresses. When his software engineering team debates whether to use

React or Vue for the frontend, the entire system design discussion remains in that project's chat history. Three weeks later, when he needs to remember why they chose React, he’s able to locate it in the conversation.

Scenario 1B: Getting Help from Teaching Assistants

Overall Objective: Communicate with teaching assistants and professors to resolve bugs and clarify assignment instructions without the delay of email.

Alex is working on a algorithms assignment due in two days and has hit a roadblock implementing a dynamic programming solution. He's stuck on understanding how to optimize his recursive function and needs clarification from the teaching assistant. In the past, he's sent emails to TAs, but responses often take 12-24 hours, and by the time he gets a reply, he's either figured it out through frustration or missed the opportunity to ask follow-up questions. He's also found that explaining complex coding problems over email is challenging since he can't easily share code snippets or have a back-and-forth conversation about his approach. Some TAs have office hours, but they conflict with his work schedule.

Alex opens the messaging app and sends a direct message to his algorithms TA, Sarah, with a description of his problem and a code snippet showing his current implementation. Within thirty minutes, Sarah responds with a suggestion about memoization and asks a clarifying question about his base cases. Alex replies with more detail, and they have a brief exchange that resolves his confusion within the next 10 minutes. The conversation is saved in his history. This communication channel allows Alex to get help and communicate efficiently without scheduling meetings or waiting for a response via email.

3.2.2 User Scenarios with Persona 2: Priya Kaur, 21 – Data Science Major

Scenario 1: Managing Multiple Project Teams

Priya is in her final semester juggling a capstone project with four team members analyzing healthcare data, a statistics study group that meets twice weekly, and a pair project for her advanced machine learning course. She's been using WhatsApp for her study groups, but the threads become chaotic when eight people are messaging simultaneously, and critical information about the dataset or meeting times gets buried. She tried creating a Slack workspace for her capstone team, but two teammates never downloaded the app, and activity amongst group members is infrequent. She recently spent twenty minutes scrolling through WhatsApp trying to find a link to a shared dataset that someone posted three weeks ago, and she's frustrated that her communication tools are creating more work instead of reducing it.

Priya creates separate chats for each of her distinct academic needs in the instant messaging app: "Capstone - Healthcare Readmission", "Stats Study Group", "ML Course - Partner Project" and a direct chat with her Professor that is writing her a recommendation letter. She invites her capstone teammates using a simple link, and they can access the chat immediately through their web browsers without downloading anything. When her team discusses which predictive model to use, she shares her analysis in the capstone chat, and everyone can refer back to that conversation when writing the final report two months later. For her statistics study group, members post problem set questions and solutions in their dedicated chat, creating a searchable repository of worked examples that Priya can reference when reviewing for exams.

Scenario 2: Building a Knowledge Repository for Technical Concepts

Priya's machine learning course covers complex algorithms and mathematical concepts that build on each other throughout the semester. She has weekly study sessions with her group, where they work through challenging problems about neural networks and optimization. When someone finally figures out how to derive the gradient for a specific loss function they send it to their WhatsApp group, however these explanations get buried under other messages. Last semester, Priya took notes during study sessions, but she's finding unable to participate in discussions and take notes, and her handwritten notes are difficult to search when she needs to review.

Priya creates a study group chat specifically for her machine learning course in the instant messaging app and encourages teammates to post explanations, worked examples directly to the chat as they discuss concepts. When her classmate Kevin finally understands and explains optimization technique to a problem during a study session, he sends a detailed message of his explanation in the chat. Two weeks later, when Priya is reviewing for the midterm and can't quite remember the details, she’s able to locate the optimization technique in the chat and finds Kevin's explanation along with the follow-up discussion. The chat history becomes a collaborative study guide that the entire group contributes to and benefits from. This creates a shared repository of information and everything is locatable.

3.2.3 User Scenarios with Persona 3: Matthew Edwards, 20 – Business Administration Major

Scenario 1: Coordinating Business Group Projects

Matthew is taking four business classes this semester, three involve group projects. In his marketing research course, he's working with three classmates on a consumer survey project. His strategic management class requires a five-person team to write a comprehensive case study analysis. He's also partnered with two students for a semester-long business plan competition requiring weekly progress meetings. He's been managing these projects through group text messages, but her marketing research group text accidentally includes the wrong classmates, her strategic management team uses a combination of texts and email, and last week she missed an important meeting location change because it was in a long thread. One teammate suggested Discord, but Matthew found it confusing, and another teammate refused to download it.

Matthew creates three distinct group chats in the instant messaging app: "MKT320 - Consumer Survey Project," "MGMT410 - Case Analysis Team," and "Business Plan Competition," inviting teammates using their university email addresses. For his marketing research project, the team discusses survey question design and coordinates data collection in their dedicated space. When they need to reference the research methodology they agreed on three weeks earlier, Matthew is able to locate the conversation. His strategic management team uses their chat to divide the case analysis into sections with clear ownership, posting draft sections for review with the entire revision process documented in one accessible location. The platform's simple, intuitive interface and web-based access allows teammates to participate from any device.

Scenario 2: Club Leadership Communication

Matthew serves as the social media coordinator for his university's entrepreneurship club, which involves working with the club's executive board and coordinating with various committee members for events. The club is planning a startup pitch competition for next month, which requires coordination between the marketing team (which Matthew leads), the logistics team handling venue and registration, and the judging committee recruiting

local entrepreneurs. Currently, the executive board uses a group text for general club business. When Matthew needs to discuss social media strategy specifically with the two students helping him, those conversations occur in the general chat and are sent to everyone.

Matthew creates several chats in the instant messaging app: "Entrepreneurship Club - Executive Board" for announcements, "Social Media Team" to communicate with the social media team, and "Pitch Competition Planning" for communicating with the logistics team organizing the upcoming event. In the social media team chat, Matthew sends weekly content calendars, shares draft Instagram posts for feedback, and discusses engagement without cluttering other conversations. For the pitch competition, the planning team uses their dedicated chat to manage the venue bookings, sponsor outreach progress, and attendance.

3.3 Epic User Stories

3.3.1 <Epic User Stories 1>

Epic 1: User Account Creation and Authentication
As a student ***I want to*** create an account, manage my profile information and securely access the application ***So that I can*** use the application to message my peers.
Acceptance Criteria ***Given*** I am a new user visiting the application for the first time ***When*** I complete the registration process with valid credentials and set my first and last name ***Then*** my account is created, my profile information is stored securely and I can log in to access the messaging features
User story 1: As a new user and student, I want to register for an account with my

university email and password so that I can create a unique identity in the app.
User story 2: As a registered user, I want to set and update my first and last name in my profile so that other users can identify me in conversations
User story 3: As a returning user, I want to log in to my account using my credentials so that I can access my existing chat rooms and message history.
User story 4: As a logged-in student user, I want to log out of my account so that my conversations remain private when I'm not using the application.
User story 5: As a user attempting to log in, I want to receive clear error messages if my credentials are incorrect so that I understand why I am unable to login.

3.3.2 <Epic User Stories 2>

Epic 2: User Account Creation and Authentication
As a student collaborating on multiple projects and involved in groups ***I want to*** create dedicated chat rooms, navigate between them, view chat rooms and delete rooms when they're no longer needed. ***So that I can*** organize my academic conversations by projects/courses and have a single platform for communication.
Acceptance Criteria ***Given*** I am a logged-in user with multiple ongoing projects ***When*** I create new chat rooms with specific names, invite members, access different chat rooms and delete rooms ***Then*** each chat room maintains separate conversation threads, names of members in the room appear when they send a message and all members can access their rooms and delete rooms for themselves from the system.
User story 1: As a student user I want to create a new chat room with a custom name so that my team has a dedicated space for project-related discussions.

User story 2: As a student user, I want to view a list of all available chat rooms that I'm a member of so that I can quickly navigate to the conversation I desire.
User story 3: As a student user, I want to select and enter specific chat rooms from my list so that I can view and participate in the conversation of that room.
User story 4: As a student user, I want to delete chat rooms that are no longer active so that I can keep my workspace organized and focused on my current activities.
User story 5: As a student user, I want to receive confirmation messages or error messages so that I know whether my action was successful or not.

3.3.3 <Epic User Stories 3>

Epic 3: Messaging in Real Time and Conversation History
As a student communicating with my classmates and study groups ***I want to*** send and receive messages instantly, and view conversation history. ***So that I can*** collaborate effectively with my peers, reference previous discussions and not lose important information.
Acceptance Criteria ***Given*** I am in an active chat room with other chat room members ***When*** I send a message or another participant sends a message ***Then*** all participants receive the message in real-time, the message is permanently stored in the conversation history and I can scroll up to view precious messages.
User story 1: As a student user in a chat room, I want to send text messages to all participants so that I can communicate information, ask questions, or share updates with my team.

User story 2: As a student user in a chat room, I want to receive messages from other participants in real-time so that I can stay updated on ongoing conversations without refreshing the page..
User story 3: As a student user, I want to view the complete conversation history via scrolling when I open a chat room so that I can catch up on messages I missed or review previous discussions.
User story 4: As a student user, I want to delete chat rooms that are no longer active so that I can keep my workspace organized and focused on my current activities.
User story 5As a student user, I want to see error messages if my message fails to send or if I lose connection to the server so that I know when my communication has been interrupted.

4 Feature List

Feature 1: User Signup/Login and Authentication	**Priority: 1**	**Effort Required: 1** (day)
Description: Users are allowed to sign up for an account with an email/username and log in to the app.		
Constraints: ***Requires MongoDB for storing user info and an API for creating/logging in to an account.***		
Comments: ***Pre-requisite for all other features***		

Feature 2: Create a group chat	**Priority: 2**	**Effort Required: 1-2** (days)
Description: Users are able to login and then create a group chat.		
Constraints: ***Requires MongoDB for storing chat and an API for creating a chat room.***		
Comments: ***Pre-requisite for adding and texting to group members.***		

Feature 3: Add peers to the chat room	**Priority: 3**	**Effort Required: 1-2**

		(days)
Description: Users are able to add other users to the group chat they made.		
Constraints: ***Requires MongoDB for storing user data and an API for adding users.***		
Comments: ***Pre-requisite for texting to group members.***		

Feature 4: Real-time Messaging in the Chat Room	**Priority: 4**	**Effort Required: 1-2** (days)
Description: Users are able to send and receive messages in the same chat room in real time.		
Constraints: ***Requires MongoDB for storing chats, WebSocket connection and an API for sending chats.***		
Comments: ***This is one of the core features of the app.***		

Feature 4: Delete group chats	**Priority: 4**	**Effort Required: 1-2** (days)
Description: Owners are able to delete group chats.		

Constraints: ***Delete group chat from MongoDB, API for deleting chat rooms.***
Comments: ***This is one of the core features of the app.***

Feature 5: Assigning multiple owners to chats.	**Priority: 5**	**Effort Required: 1-2** (days)
Description: Allow multiple owners/moderators for a chat room.		
Constraints: ***Add permission data to MongoDB, API for giving owner permission to users.***		
Comments:		

Feature 6: Edit/delete messages.	**Priority: 6**	**Effort Required: 1-2** (days)
Description: Allow users to delete or edit their messages.		
Constraints: ***Modify data in MongoDB, API for editing messages.***		
Comments:		

Feature 7: Allow file share (pictures, documents).	**Priority: 7**	**Effort Required: 1-2** (days)
Description: Allow users to send files and images into the group chat.		
Constraints: ***Storage in MongoDB.***		
Comments:		

Feature 8: Allow Users to Block other Users.	**Priority: 8**	**Effort Required: 1-2** (days)
Description: A “Block” button to allow users to block another.		
Constraints: ***Unable to DM to this user.***		
Comments:		

Feature 9: Allow Profile Customization.	**Priority: 9**	**Effort Required: 1-2** (days)
Description: Allow users to add a profile picture, bio and update their name.		
Constraints:		

Database storage.
Comments: Making the platform more welcoming/personal.

Feature 10: Push notifications to email.	**Priority: 10**	**Effort Required: 1-2** (days)
Description: When a message is sent or a user is invited to a new chat room, a push notification should be sent via email.		
Constraints: ***Authentication.***		
Comments: Making the platform more engaging.		

Feature 11: Search	**Priority: 11**	**Effort Required: 1-2** (days)
Description: User is able to search their chat rooms for specific content, and the platform takes them to the message with that exact content.		
Comments: Making the platform easier to use and locate important information.		

Feature 12: Direct DMs.	**Priority: 12**	**Effort Required: 1-2** (days)
Description: Allow users to message a single user directly.		

Comments: Making the platform easier to use and locate important information.